FACULTY OF ENGINEERING

REII 327: DESIGN 2026

Sub-Project Report: REII nRF24L01 Link Bring-Up & Characterisation

By:

Riekert Nel

40496325

Submitted in pursuit of the degree

BACHELOR OF ENGINEERING

In

COMPUTER ELECTRONIC ENGINEERING

North-West University Potchefstroom Campus

Abstract

The primary objective of this sub-project was to systematically verify the reliability of an nRF24L01+ RF link prior to committing resources to a custom STM32F411 dongle and application protocol. Testing evaluated hardware stability, channel interference, and optimal data rates for telemetry. This report exclusively details line-of-sight (LOS) packet-loss testing and characterisation. While extreme-range stress tests confirmed hardware stability up to 190 meters LOS, the final configuration is optimized for the project's 5 to 10-meter operational envelope. Operating at 2000 kbps ensures ultra-low latency and maximum interference evasion, provided specific hardware decoupling modifications are applied.

Table of Contents

Abstract 2
1. Problem Statement 5
2. System Setup: Hardware Bring-Up and Software Methods 5
2.1 Method (Software Setup) 5
2.2 Implementation (Hardware Bring-Up) 5
3. Deliverable 1: Packet-Loss vs. Distance (LOS) 7
3.1 Implementation and Results 7
3.2 Discussion 7
3.3 Conclusion 7
4. Deliverable 2: Channel Selection and Interference Analysis 8
4.1 Method and Implementation 8
4.2 Results and Discussion 8
4.3 Conclusion 9
5. Deliverable 3: Recommended nRF24 Configuration 9
5.1 Discussion on Retry Settings (Latency vs. Reliability) 9
5.2 Conclusion: Final Configuration Table 9
6. Design Communication Protocol for Future Usage 10
Appendix 11
A1: Retry Statistics and ACK Reliability 11
A2: Hardware Wiring Schematics 12
B1: run_test.py 13
B2: channel_analysis.py 16

List of Figure

Figure 1: Physical hardware setup 6
Figure 2: Packet-loss versus distance curves (LOS) 7
Figure 3: Channel survey spectrum analysis and Wi-Fi AP interference overlay 8
Figure 4: ACK Reliability vs distance 11
Figure 5: RX Module (nodemcu32s) wiring diagram 12
Figure 6: TX Module (esp32c3) wiring diagram 12

List of Tables

Table 1: 5-Meter Interference Benchmark (PA_MAX, NOACK Mode) 9
Table 2: Recommended nRF24 Configuration 9

1. Problem Statement

The remote-control dashboard project requires a proven, highly reliable RF communication link. Before finalizing the custom ground-station dongle and the underlying communication protocol, the nRF24L01+ radio modules must be characterized. The core deliverables required to validate this link are packet-loss versus distance curves at multiple data rates, a channel selection strategy based on real-world interference analysis, and a final recommended nRF24 configuration defining the channel, data rate, power, and retry settings.

2. System Setup: Hardware Bring-Up and Software Methods

To execute the required characterisation, a standardized test bench was established incorporating both microcontroller hardware and PC-based automation software.

2.1 Method (Software Setup)

Instead of relying on manual packet counting, the test methodology utilized a suite of Python automation scripts (run_test.py and channel_analysis.py) executed from a host PC. The PC communicates over a serial port to the receiver (RX) node, which acts as a master controller. The RX node utilizes a fixed, high-power control link (Channel 80) to command the mobile transmitter (TX) node to initiate test bursts using specific configurations. The Python scripts parse the serial telemetry, automatically log the packet error rates (PER) to CSV files, and generate the required performance graphs.

2.2 Implementation (Hardware Bring-Up)

The physical architecture deployed an ESP32-C3 development board for the mobile TX node (powered by a USB battery bank) and a NodeMCU-32S for the ground-station RX node. During initial testing, the system suffered consistent brownouts because the PA+LNA modules draw current bursts exceeding 115 mA when operating at maximum transmit power (PA_MAX). This was resolved by implementing a mandatory hardware modification: a 10 µF electrolytic capacitor and a 100 nF ceramic capacitor were added directly across the VCC and GND pins of both nRF24 modules.

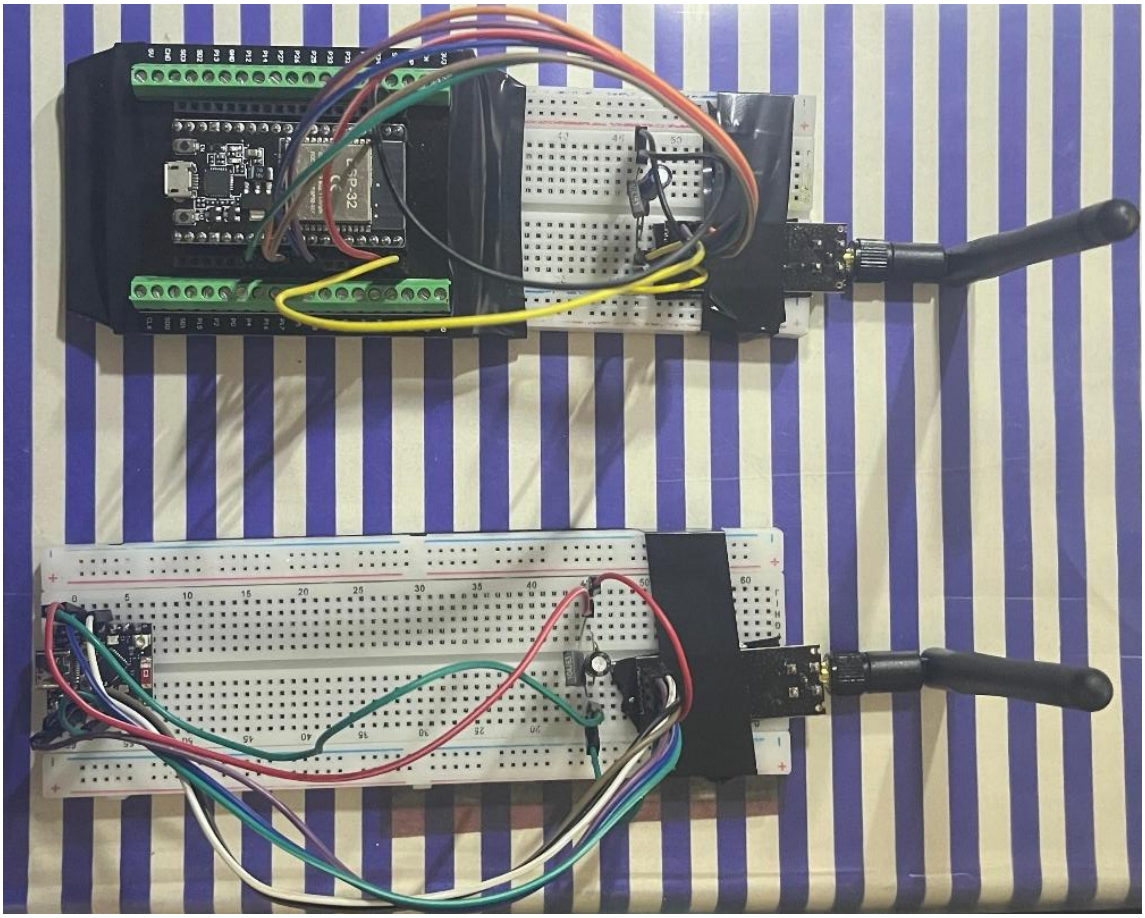


Figure 1: Physical hardware setup

Figure 1 provides visual proof of the final hardware bring-up, specifically highlighting the decoupling capacitors required to stabilize the 3.3V rail. Additionally, bench testing demonstrated that placing the antennas closer than 1 meter at PA_MAX saturates the receiving Low Noise Amplifier (LNA), requiring the use of PA_LOW for short-range verification.

3. Deliverable 1: Packet-Loss vs. Distance (LOS)

To determine the effective range and optimum transmission speed, the link was systematically tested over varying distances with unobstructed line-of-sight.

3.1 Implementation and Results

A distance sweep was conducted from 1 meter up to 190 meters LOS. At each distance marker, the Python script commanded the TX node to transmit 1,000 packets at PA_MAX across three data rates: 250 kbps, 1000 kbps, and 2000 kbps. The test measured both raw physical-layer packet loss (NOACK mode) and effective delivery after hardware retries (ACK mode).

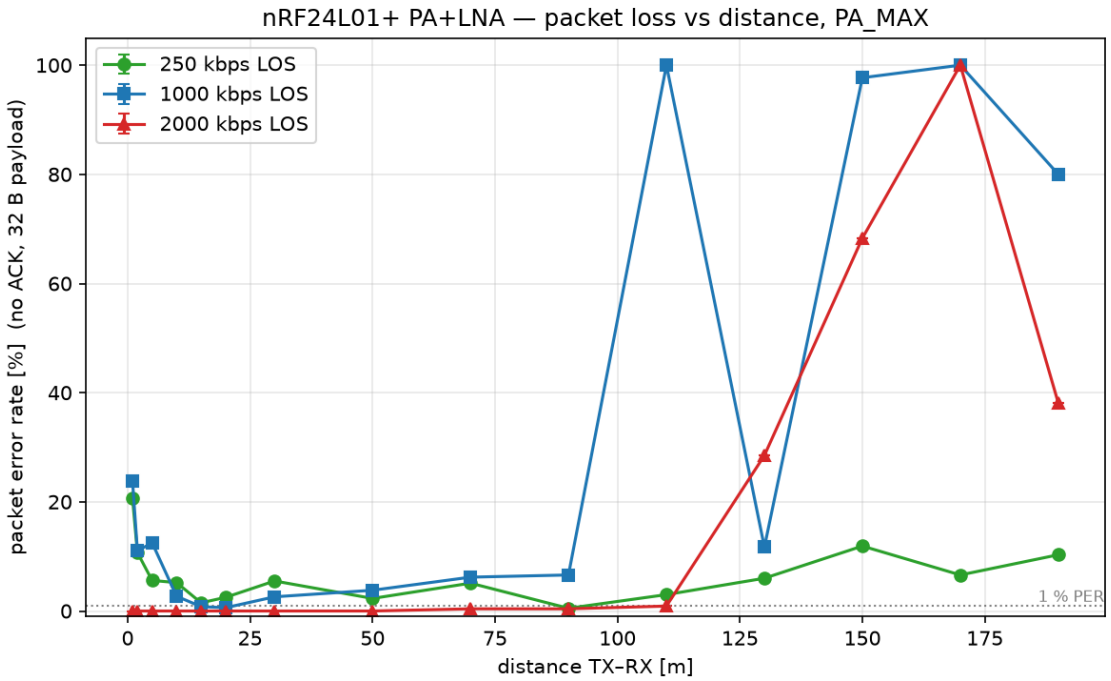


Figure 2: Packet-loss versus distance curves (LOS)

Figure 2 illustrates the raw packet error rate (PER) over distance for the three tested data rates, demonstrating the inverse relationship between data rate and range sensitivity.

3.2 Discussion

The empirical data clearly maps the performance trade-offs of the three data rates. While the 250 kbps configuration exhibited superior sensitivity at extreme ranges (maintaining a connection up to 190m), it suffered slightly higher raw packet loss at short ranges (5.60% PER at 5m). Conversely, the 2000 kbps rate degraded past 100 meters, but delivered absolutely flawless physical-layer performance at short range, logging a 0.00% raw PER at both 5m and 10m.

3.3 Conclusion

The final application requires the mobile node to control a machine on campus from a typical standoff distance of 5 to 10 meters. Therefore, **2000 kbps** is the definitive choice for the data rate. At this speed, the packet spends significantly less time on the air, reducing the probability of Wi-Fi collisions. The 190-meter stress tests successfully proved the hardware decoupling is stable under maximum load, but the configuration must be optimized for the 10-meter operational envelope.

4. Deliverable 2: Channel Selection and Interference Analysis

Coexistence within the 2.4 GHz ISM band requires strategic channel selection to avoid interference from standard 802.11b/g/n Wi-Fi networks and other project groups.

4.1 Method and Implementation

Channel selection was automated via the channel_analysis.py script. The RX node sequentially sweeps all 126 nRF channels to measure ambient RF energy (carrier-detect) while simultaneously performing a standard Wi-Fi access point scan. The script calculates a penalty score for each channel based on a weighted overlap with detected Wi-Fi channels (±11 MHz bandwidth) and ambient RF hits, prioritizing quiet gaps.

4.2 Results and Discussion

During testing, the primary campus operating environment yielded 6 overlapping Wi-Fi networks, primarily clustered around Wi-Fi channels 2, 6, and 11. The Python analysis identified a quiet spectral gap at nRF Channel 77 (2477 MHz), which is completely free from high-RSSI Wi-Fi overlap as it sits above the primary Wi-Fi bands.

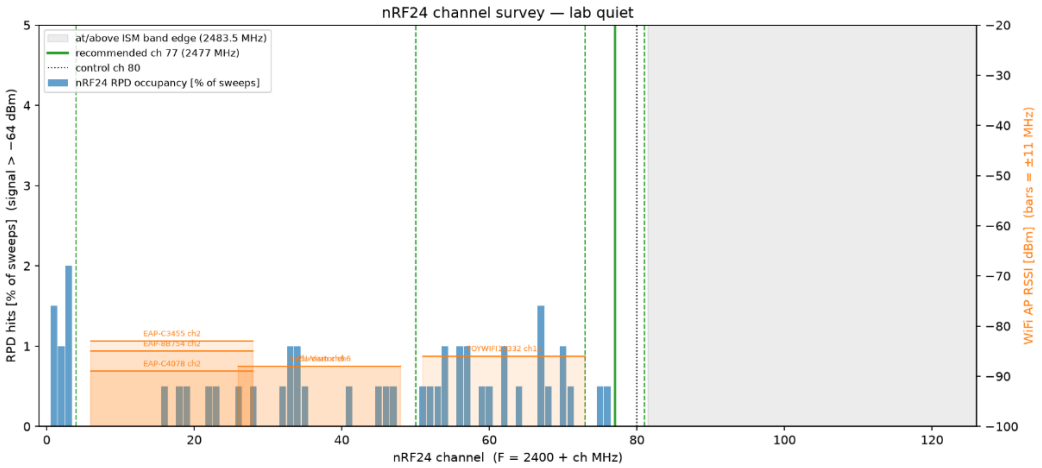


Figure 3: Channel survey spectrum analysis and Wi-Fi AP interference overlay

Figure 3 graphs the ambient RF energy alongside the detected Wi-Fi networks, visually confirming the algorithm's recommendation of Channel 77. To ensure multi-group coexistence, a minimum guard band of 4 nRF channels must be maintained between active REII links to prevent adjacent-channel interference. The algorithm generated a safe group plan of channels 4, 50, 73, 77, and 81.

To quantify the impact of poor channel selection, a deliberate interference benchmark was run at 5 meters. Testing on nRF Channel 11—which sits directly inside the active Wi-Fi Channel 2 envelope—yielded a raw physical-layer packet error rate of 17.20% at 250 kbps and 20.10% at 1 Mbps. When tested on a cleaner frequency (nRF Channel 4), the raw packet loss dropped to 7.50% and 12.70% respectively. This proves that operating within a Wi-Fi band severely degrades the physical RF link, necessitating the hardware Auto-ACK to work harder to maintain delivery.

Table 1: 5-Meter Interference Benchmark (PA_MAX, NOACK Mode)

nRF Channel	Environment	250 kbps PER	1000 kbps PER	2000 kbps PER
Channel 11	Congested (Inside Wi-Fi Ch 2)	17.20%	20.10%	0.00%
Channel 4	Quiet Spectral Gap	7.50%	12.70%	0.50%

4.3 Conclusion

Based on the environmental sweep and interference benchmarking, **nRF Channel 77 (2477 MHz)** is selected as the primary operating frequency. It successfully evades campus Wi-Fi interference while remaining within the legal ISM band.

5. Deliverable 3: Recommended nRF24 Configuration

The final configuration consolidates the empirical findings into a definitive register map for the upcoming STM32F411 integration.

5.1 Discussion on Retry Settings (Latency vs. Reliability)

By optimizing the system for 2000 kbps, the latency footprint is drastically reduced. At 2 Mbps with a 32-byte payload, a single transmission attempt (T_frame) takes approximately 0.16 ms. Furthermore, the Auto Retry Delay (ARD) can be safely reduced from 1500 µs to 500 µs at this speed. A full transmission cycle (Frame + Delay + ACK) requires only ~0.82 ms. Setting a maximum limit of 5 retries (ARC = 5) caps the absolute worst-case radio delay at a mere 4.1 ms. This sits comfortably an order of magnitude below the 50 ms dashboard requirement, ensuring instantaneous machine control. The empirical basis for this retry limit is provided in Appendix A, which demonstrates that retries rarely exceed 1 to 2 attempts at short range.

5.2 Conclusion: Final Configuration Table

Table 2 outlines the mandated hardware parameters derived from the characterisation phase, optimized for the final application range.

Table 2: Recommended nRF24 Configuration

Parameter	Selected Value	Justification
Channel (RF_CH)	**77 (2477 MHz)**	Located in a quiet spectral gap (74–81) strictly above the congested campus Wi-Fi infrastructure (Wi-Fi channels 1, 6, 11).
Data Rate (RF_DR)	**2000 kbps**	Achieved 0.00% raw physical packet loss at the operational 5–10 meter range; significantly minimizes time-on-air to evade Wi-Fi collisions.
PA Level (RF_PWR)	**PA_MAX**	Verified stable under load with decoupling capacitors (must be reduced to PA_LOW for < 2m bench tests).
Retry Count (ARC)	**5**	Safely absorbs transient interference while strictly capping the worst-case radio latency at ~4.1 ms.
Retry Delay (ARD)	**500 µs**	Hardware constraint; optimized for 2000 kbps for faster recovery than the 1500 µs required for 250 kbps.
Address/CRC	**5 bytes / 16-bit**	Standard robustness configuration.

6. Design Communication Protocol for Future Usage

The final objective of this phase is to dictate how the STM32F411 and application protocol will interact with the RF link.

Based on the highly successful ACK mode performance, implementing a complex Automatic Repeat Request (ARQ) scheme at the software application level is deemed unnecessary and constitutes over-engineering. The nRF24L01+'s intrinsic Enhanced ShockBurst hardware Auto-ACK feature is perfectly capable of maintaining link integrity, successfully masking physical-layer drops under extreme stress conditions and delivering flawless 0.00% packet loss at the operational 10-meter range.

The future STM32 firmware should rely strictly on the hardware configuration defined in Table 2. The communication protocol will utilize standard data packets for downstream commands from the dashboard, and leverage ACK-payloads (piggybacking data onto the automatic hardware acknowledgments) for upstream telemetry from the mobile node. This approach minimizes overhead, prevents unnecessary application-layer timeouts, and officially validates the RF hardware architecture for the next phase of development.

Appendix

A1: Retry Statistics and ACK Reliability

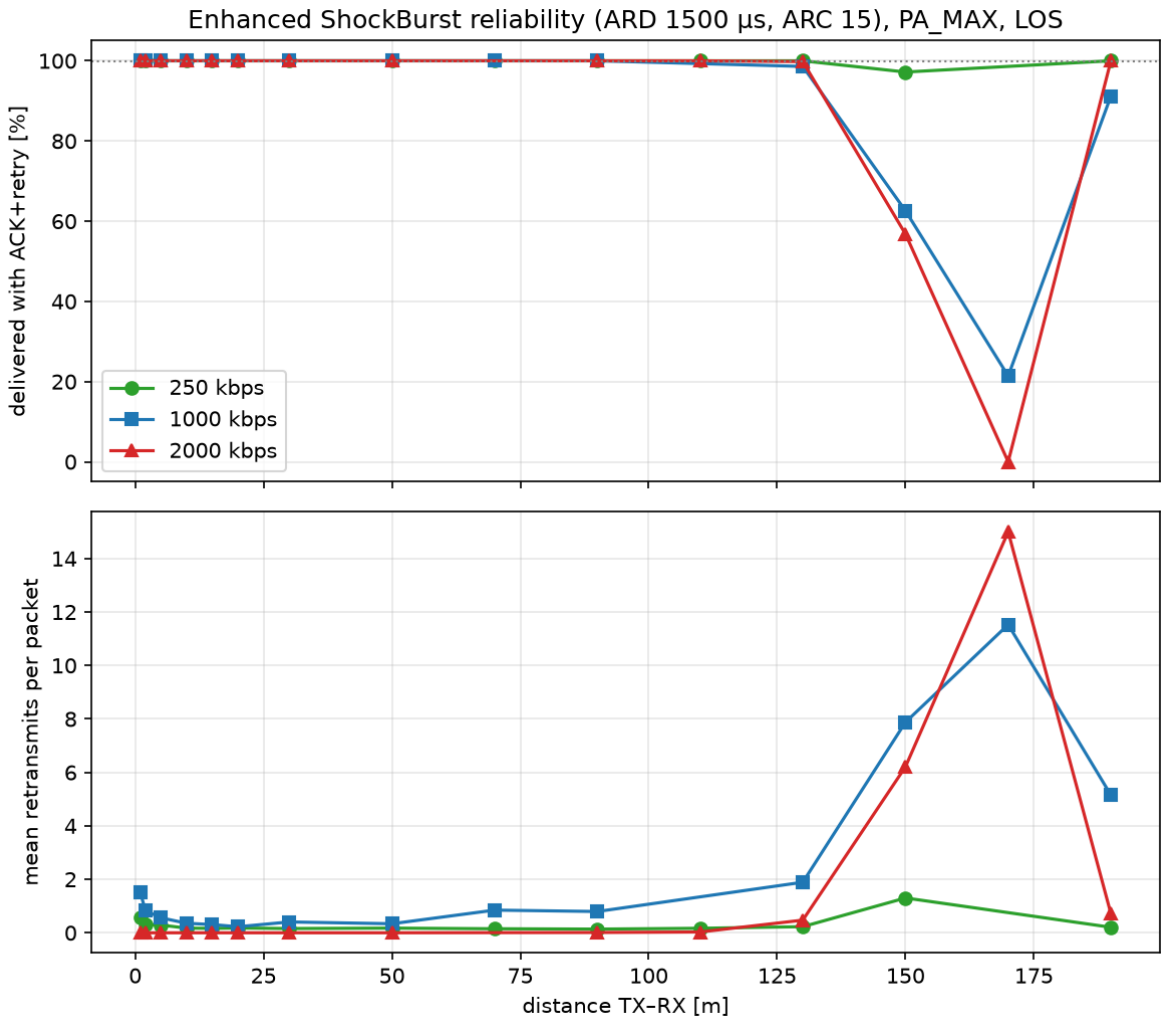


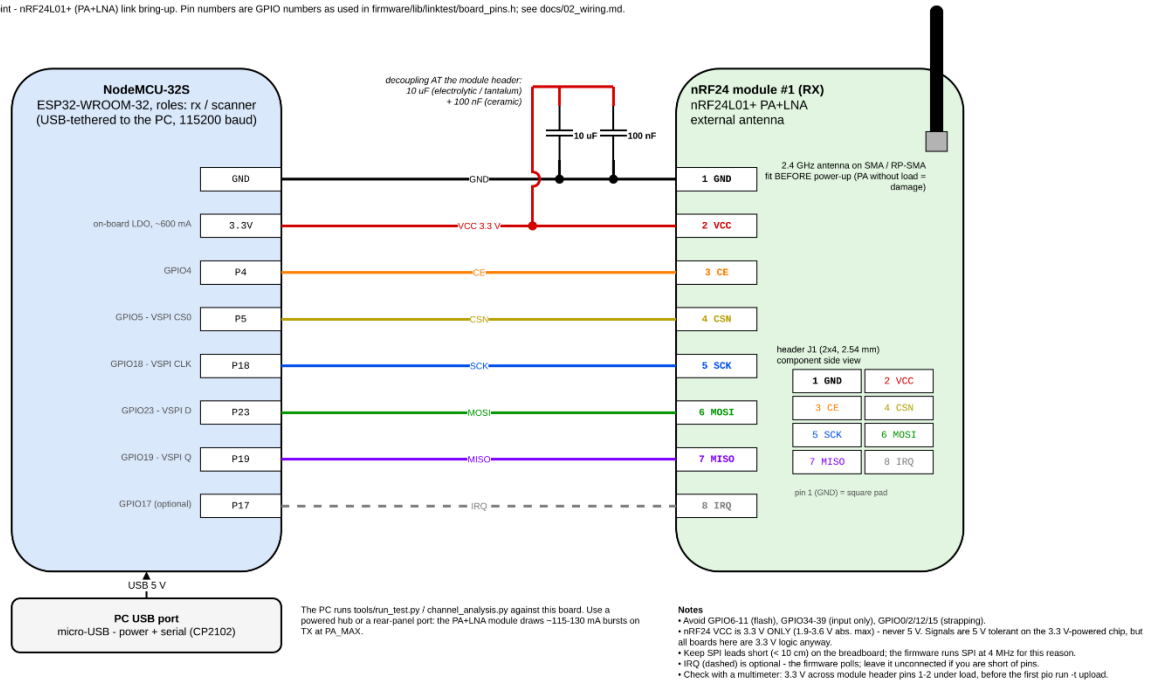
Figure 4: ACK Reliability vs distance

Figure 4 charts the mean retries per packet over distance, confirming that the ARC = 5 limit provides a sufficient buffer for reliable transmission at operational ranges.

A2: Hardware Wiring Schematics

Receiver / ground-station stand-in: NodeMCU-32S + nRF24L01+ PA+LNA

REII checkpoint - nRF24L01+ (PA+LNA) link bring-up. Pin numbers are GPIO numbers as used in firmware/lib/linktest/board_pins.h; see docs/02_wiring.md.



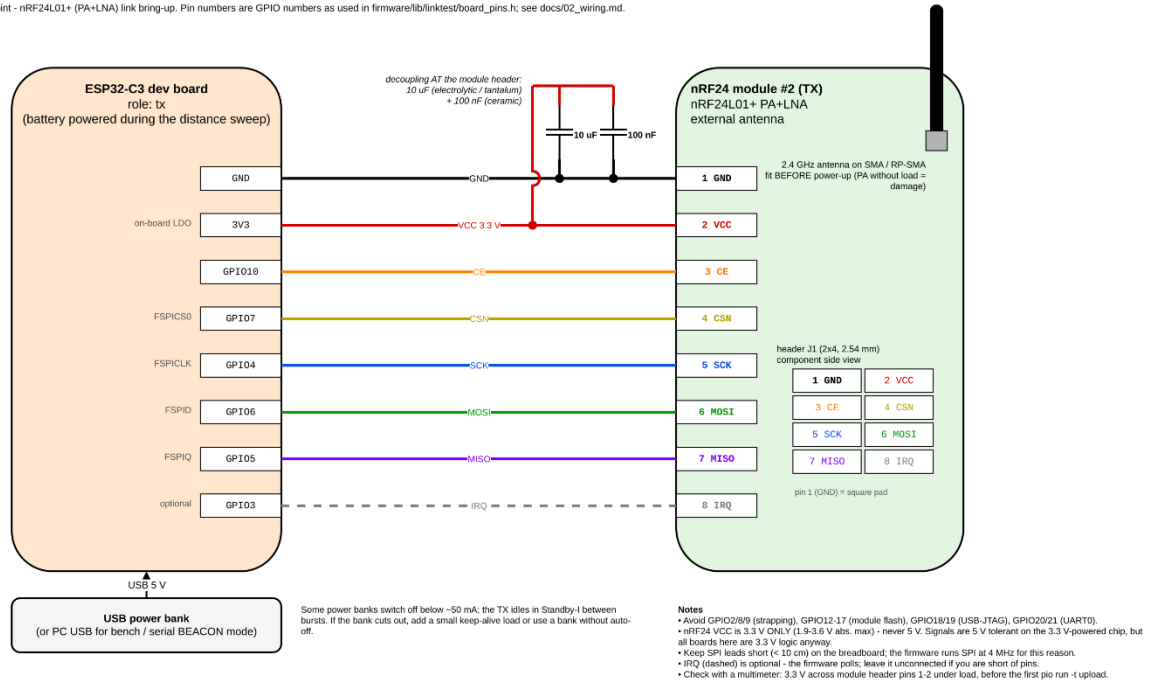
The PC runs tools/run_test.py / channel_analysis.py against this board. Use a powered hub or a rear-panel port: the PA+LNA module draws ~115-130 mA bursts on TX at PA_MAX.

Notes
- Avoid GPIO6-11 (flash), GPIO34-39 (input only), GPIO0/2/12/15 (strapping).
- nRF24 VCC is 3.3 V ONLY (1.9-3.6 V abs. max) - never 5 V. Signals are 5 V tolerant on the 3.3 V-powered chip, but all boards here are 3.3 V logic anyway.
- Keep SPI leads short (< 10 cm) on the breadboard; the firmware runs SPI at 4 MHz for this reason.
- IRQ (dashed) is optional - the firmware polls; leave it unconnected if you are short of pins.
- Check with a multimeter: 3.3 V across module header pins 1-2 under load, before the first pio run -t upload.

Figure 5: RX Module (nodemcu32s) wiring diagram

Transmitter / mobile node: ESP32-C3 dev board + nRF24L01+ PA+LNA

REII checkpoint - nRF24L01+ (PA+LNA) link bring-up. Pin numbers are GPIO numbers as used in firmware/lib/linktest/board_pins.h; see docs/02_wiring.md.



Some power banks switch off below ~50 mA; the TX idles in Standby-I between bursts. If the bank cuts out, add a small keep-alive load or use a bank without auto-off.

Notes
- Avoid GPIO2/8/9 (strapping), GPIO12-17 (module flash), GPIO18/19 (USB-JTAG), GPIO20/21 (UART0).
- nRF24 VCC is 3.3 V ONLY (1.9-3.6 V abs. max) - never 5 V. Signals are 5 V tolerant on the 3.3 V-powered chip, but all boards here are 3.3 V logic anyway.
- Keep SPI leads short (< 10 cm) on the breadboard; the firmware runs SPI at 4 MHz for this reason.
- IRQ (dashed) is optional - the firmware polls; leave it unconnected if you are short of pins.
- Check with a multimeter: 3.3 V across module header pins 1-2 under load, before the first pio run -t upload.

Figure 6: TX Module (esp32c3) wiring diagram

B1: run_test.py

```
#!/usr/bin/env python3
"""Drive one distance point of the packet-loss characterisation.

Talks to the RX board (role rx) over serial, runs NOACK + ACK bursts for every requested
data rate and appends one CSV row per burst to results/packet_loss_<date>.csv.

    python tools/run_test.py --port /dev/ttyUSB0 --distance 5 --los 1 --pa MAX --n 1000
    python tools/run_test.py --port COM7 --distance 1 --pa LOW --rates 1000 --n 200      #
smoke test
    python tools/run_test.py --port COM7 --distance 8 --los 0 --note "LOS" --channel 76
"""

from __future__ import annotations

import argparse
import csv
import datetime as dt
import pathlib
import sys

from linktest_serial import LinkTestSerial

HERE = pathlib.Path(__file__).resolve().parent
RESULTS = HERE.parent / "results"

CSV_COLUMNS = [
    "timestamp", "distance_m", "los", "note", "channel", "rate_kbps", "pa", "mode", "n",
    "tx_sent", "rx_unique", "rx_dup", "rx_ooo", "tx_acked", "tx_failed", "retries_total",
    "retries_max", "rpd_hits", "rpd_samples", "duration_ms", "per_pct", "status",
]

# RESULT,run_id,mode,rate_kbps,ch,pa,n,tx_sent,rx_unique,dup,ooo,tx_acked,tx_failed,
#        retries_total,retries_max,rpd_hits,rpd_samples,duration_ms,status
RESULT_FIELDS = [
    "run_id", "mode", "rate_kbps", "channel", "pa", "n", "tx_sent", "rx_unique", "rx_dup",
    "rx_ooo", "tx_acked", "tx_failed", "retries_total", "retries_max", "rpd_hits",
    "rpd_samples", "duration_ms", "status",
]


def parse_result(line: str) -> dict[str, str]:
    parts = line.split(",")
    if parts[0] != "RESULT" or len(parts) != len(RESULT_FIELDS) + 1:
        raise ValueError(f"malformed RESULT line: {line!r}")
    return dict(zip(RESULT_FIELDS, parts[1:]))


def per_percent(r: dict[str, str]) -> float:
    n = int(r["n"])
    status = r["status"]
    if n == 0 or status == "RATE_UNSUPPORTED":
        return float("nan")
    if status == "CTRL_TIMEOUT":
        return 100.0
    if r["mode"] == "ACK" and status in ("OK", "TX_ABORTED"):
        # unsent packets after an abort count as failed
        return 100.0 * (1 - int(r["tx_acked"]) / n)
    return 100.0 * (1 - int(r["rx_unique"]) / n)


def burst_timeout_s(mode: str, n: int, spacing_us: int) -> float:
    # Mirrors firmware: ctrl retry 2 s + burst + report wait 5 s + margin.
    if mode == "ACK":
        return 2 + n * 0.050 + 1 + 5 + 3
    return 2 + n * spacing_us / 1e6 + 1.5 + 5 + 3


def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", required=True, help="serial port of the RX board")
    ap.add_argument("--distance", type=float, required=True, help="TX-RX distance in metres")
    ap.add_argument("--los", type=int, default=1, choices=(0, 1), help="1 = line of sight, 0 =
obstructed")
```

```
    ap.add_argument("--note", default="", help="free text, e.g. 'through wall'")
    ap.add_argument("--channel", type=int, default=76, help="RF_CH for the bursts (0..125)")
    ap.add_argument("--rates", default="250,1000,2000", help="comma list of kbps values")
    ap.add_argument("--pa", default="MAX", choices=("MIN", "LOW", "HIGH", "MAX"))
    ap.add_argument("--modes", default="NOACK,ACK", help="comma list of NOACK / ACK")
    ap.add_argument("--n", type=int, default=1000, help="packets per burst (<= 2000)")
    ap.add_argument("--spacing-us", type=int, default=2000, help="NOACK packet spacing")
    ap.add_argument("--repeat", type=int, default=1, help="repeat every burst this many
times")
    ap.add_argument("--out", default=None, help="CSV file (default
results/packet_loss_<date>.csv)")
    ap.add_argument("--echo", action="store_true", help="print raw serial traffic to stderr")
    args = ap.parse_args()

    if not 1 <= args.n <= 2000:
        ap.error("--n must be 1..2000")
    if not 0 <= args.channel <= 125:
        ap.error("--channel must be 0..125")

    RESULTS.mkdir(exist_ok=True)
    out = pathlib.Path(args.out) if args.out else RESULTS /
f"packet_loss_{dt.date.today().isoformat()}.csv"
    new_file = not out.exists()

    link = LinkTestSerial(args.port, echo=args.echo)
    if not link.wait_ready():
        print("RX board not answering (is role rx flashed? right port?)", file=sys.stderr)
        return 2

    link.send("PING")
    pong = link.wait_for(("PONG",), 4.0)
    if not pong or pong.split(",")[1] != "1":
        print("TX not reachable over the control link - check TX power / distance",
file=sys.stderr)
        print("Continuing anyway; rows will be logged as CTRL_TIMEOUT.", file=sys.stderr)

    rates = [int(r) for r in args.rates.split(",") if r]
    modes = [m.strip().upper() for m in args.modes.split(",") if m]
    rows: list[dict[str, object]] = []
    with out.open("a", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=CSV_COLUMNS)
        if new_file:
            w.writeheader()
        for _ in range(args.repeat):
            for rate in rates:
                for mode in modes:
                    cmd = f"TEST {mode} {rate} {args.channel} {args.pa} {args.n}
{args.spacing_us}"
                    link.send(cmd)
                    line = link.wait_for(("RESULT", "ERR"), burst_timeout_s(mode, args.n,
args.spacing_us))
                    if line is None:
                        print(f"  {mode:5s} {rate:>4d} kbps: no RESULT (serial timeout)",
file=sys.stderr)
                        continue
                    if line.startswith("ERR"):
                        print(f"  firmware rejected '{cmd}': {line}", file=sys.stderr)
                        continue
                    r = parse_result(line)
                    row: dict[str, object] = {
                        "timestamp": dt.datetime.now().isoformat(timespec="seconds"),
                        "distance_m": args.distance,
                        "los": args.los,
                        "note": args.note,
                        **{k: r[k] for k in CSV_COLUMNS if k in r},
                        "per_pct": f"{per_percent(r):.2f}",
                    }
                    w.writerow(row)
                    fh.flush()
                    rows.append(row)
                    print(
                        f"  {mode:5s} {rate:>4d} kbps ch{r['channel']} PA_{r['pa']}: "
                        f"PER={row['per_pct']}%  rx_unique={r['rx_unique']}/{r['n']}  "
                        f"acked={r['tx_acked']} failed={r['tx_failed']}
retries={r['retries_total']} "
                        f"(max {r['retries_max']})  rpd={r['rpd_hits']}/{r['rpd_samples']}
{r['status']}"
```

```
                    )
    link.close()
    print(f"{len(rows)} rows appended to {out}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

B2: channel_analysis.py

```
#!/usr/bin/env python3
"""Channel survey: capture the scanner firmware output, score every nRF24 channel, recommend.

Capture from the board (role scanner flashed on the NodeMCU-32S):
    python tools/channel_analysis.py --port /dev/ttyUSB0 --sweeps 200 --label

Re-analyse a saved capture:
    python tools/channel_analysis.py --infile results/channel_survey_2026-09-10_1432.raw

Outputs (results/):
    channel_survey_<stamp>.raw   verbatim serial lines (RPD,... / WIFI,... )
    channel_survey_<stamp>.csv   per-channel table: ch, freq_mhz, rpd_occupancy, wifi_penalty,
score, in_ism
    channel_survey_<stamp>.png   occupancy bars + WiFi overlay + recommendation
and prints the top channels plus a multi-group channel plan.

"""

from __future__ import annotations

import argparse
import datetime as dt
import pathlib
import sys

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

HERE = pathlib.Path(__file__).resolve().parent
RESULTS = HERE.parent / "results"

NUM_CH = 126
ISM_BOTTOM_CH = 2              # keep a 2 MHz-wide (2 Mbps) signal inside 2400.0-2483.5 MHz
ISM_TOP_CH = 81
WIFI_HALF_BW_MHZ = 11.0        # 20/22 MHz channel
CONTROL_CH = 80                # firmware control channel (protocol.h CTRL_CHANNEL); excluded
from the recommendation


def wifi_center_mhz(wifi_ch: int) -> float:
    if wifi_ch == 14:
        return 2484.0
    return 2407.0 + 5.0 * wifi_ch


def nrf_freq_mhz(ch: int) -> float:
    return 2400.0 + ch


def capture(port: str, sweeps: int) -> list[str]:
    from linktest_serial import LinkTestSerial

    link = LinkTestSerial(port)
    link.send(f"SCAN {sweeps}")
    lines: list[str] = []
    got_start = False
    t_limit = 60 + sweeps * 126 * 0.0005 * 1.5  # dwell + SPI overhead, generous
    import time

    t0 = time.monotonic()
    while time.monotonic() - t0 < t_limit:
        line = link.readline()
        if line is None:
            continue
        if line.startswith("SCAN_START"):
            got_start = True
            lines = []
            continue
        if not got_start:
            continue
        if line == "END":
```

```
            break
        if line.startswith(("RPD,", "WIFI,")):
            lines.append(line)
    link.close()
    if not lines:
        raise SystemExit("no scanner output - is the scanner role flashed?")
    return lines


def parse(lines: list[str]):
    rpd = np.zeros(NUM_CH)
    sweeps = 1
    wifi: list[tuple[str, str, int, int]] = []
    for line in lines:
        parts = line.strip().split(",")
        if parts[0] == "RPD" and len(parts) == 4:
            ch, hits, sw = int(parts[1]), int(parts[2]), int(parts[3])
            if 0 <= ch < NUM_CH:
                rpd[ch] = hits
                sweeps = max(sweeps, sw)
        elif parts[0] == "WIFI" and len(parts) >= 5:
            ssid, bssid, ch, rssi = parts[1], parts[2], int(parts[3]), int(parts[4])
            wifi.append((ssid, bssid, ch, rssi))
    return rpd / sweeps, wifi, sweeps


def wifi_penalty(wifi) -> np.ndarray:
    pen = np.zeros(NUM_CH)
    freqs = np.array([nrf_freq_mhz(c) for c in range(NUM_CH)])
    for _ssid, _bssid, wch, rssi in wifi:
        centre = wifi_center_mhz(wch)
        weight = 10 ** ((min(max(rssi, -100), -30) + 100) / 20.0)  # -100 dBm -> 1, -40 dBm ->
1000
        d = np.abs(freqs - centre)
        # flat inside +-9 MHz, raised-cosine roll-off to 0 at +-11 MHz (a bit beyond to be
safe)
        overlap = np.where(d <= 9.0, 1.0, np.where(d >= 13.0, 0.0, 0.5 * (1 + np.cos(np.pi *
(d - 9.0) / 4.0))))
        pen += weight * overlap
    return pen


def score_channels(rpd_occ: np.ndarray, wifi):
    wp = wifi_penalty(wifi)
    wp_norm = wp / wp.max() if wp.max() > 0 else wp
    score = 0.5 * wp_norm + 0.5 * rpd_occ
    return score, wp


def guard_ok(ch: int, score: np.ndarray, guard: int, thresh: float, top_ch: int) -> bool:
    lo, hi = ch - guard, ch + guard
    if lo < ISM_BOTTOM_CH or hi > top_ch:
        return False
    return bool(np.all(score[lo:hi + 1] <= thresh))


def recommend(score: np.ndarray, guard: int, plan: int, allow_above_ism: bool):
    top_ch = NUM_CH - 1 if allow_above_ism else ISM_TOP_CH
    cand = [c for c in range(ISM_BOTTOM_CH, top_ch + 1) if c != CONTROL_CH]
    ranked = sorted(cand, key=lambda c: (score[c], c))
    best10 = ranked[:10]

    # single-link pick: the channel itself and +-half the group spacing must all be in the
quiet 30 %
    half = max(1, guard // 2)
    thresh = np.percentile(score[ISM_BOTTOM_CH: top_ch + 1], 30)
    guarded = [c for c in ranked if guard_ok(c, score, half, thresh, top_ch)]
    best_guarded = guarded[0] if guarded else ranked[0]

    # multi-group plan: greedily pick channels >= `guard` apart, quietest first
    plan_chs: list[int] = []
    for c in ranked:
        if all(abs(c - p) >= guard for p in plan_chs):
            plan_chs.append(c)
        if len(plan_chs) >= plan:
            break
    return best10, best_guarded, sorted(plan_chs), bool(guarded), half
```

```
def plot(rpd_occ, wifi, score, best, plan_chs, out_png: pathlib.Path, label: str):
    fig, ax = plt.subplots(figsize=(12, 5.5))
    chs = np.arange(NUM_CH)
    ax.bar(chs, rpd_occ * 100, width=0.9, color="tab:blue", alpha=0.7, label="nRF24 RPD
occupancy [% of sweeps]")
    ax.set_xlabel("nRF24 channel  (F = 2400 + ch MHz)")
    ax.set_ylabel("RPD hits [% of sweeps]  (signal > -64 dBm)")
    ax.set_xlim(-1, NUM_CH)
    ax.set_ylim(0, max(5, float(rpd_occ.max() * 100) * 1.15))
    ax.axvspan(ISM_TOP_CH + 0.5, NUM_CH, color="grey", alpha=0.15, label="at/above ISM band
edge (2483.5 MHz)")

    ax2 = ax.twinx()
    for ssid, _bssid, wch, rssi in wifi:
        c = wifi_center_mhz(wch) - 2400
        ax2.add_patch(plt.Rectangle((c - WIFI_HALF_BW_MHZ, -100), 2 * WIFI_HALF_BW_MHZ, rssi +
100,
                                    color="tab:orange", alpha=0.12))
        ax2.plot([c - WIFI_HALF_BW_MHZ, c + WIFI_HALF_BW_MHZ], [rssi, rssi],
color="tab:orange", lw=1)
        ax2.text(c, rssi + 1, f"{ssid[:12]} ch{wch}", ha="center", fontsize=6,
color="tab:orange")
    ax2.set_ylim(-100, -20)
    ax2.set_ylabel("WiFi AP RSSI [dBm]  (bars = ±11 MHz)", color="tab:orange")

    ax.axvline(best, color="tab:green", lw=2, label=f"recommended ch {best}
({nrf_freq_mhz(best):.0f} MHz)")
    for p in plan_chs:
        if p != best:
            ax.axvline(p, color="tab:green", lw=1, ls="--")
    ax.axvline(CONTROL_CH, color="k", lw=1, ls=":", label=f"control ch {CONTROL_CH}")
    ax.set_title(f"nRF24 channel survey — {label}")
    ax.legend(loc="upper left", fontsize=8)
    fig.tight_layout()
    fig.savefig(out_png, dpi=150)


def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    src = ap.add_mutually_exclusive_group(required=True)
    src.add_argument("--port", help="serial port of the scanner board")
    src.add_argument("--infile", help="re-analyse a saved .raw capture")
    ap.add_argument("--sweeps", type=int, default=200)
    ap.add_argument("--label", default="", help="free text for the plot title / filename")
    ap.add_argument("--guard", type=int, default=4, help="min channel spacing between groups")
    ap.add_argument("--plan", type=int, default=5, help="number of simultaneous links to plan
for")
    ap.add_argument("--allow-above-ism", action="store_true", help="also consider channels
84..125")
    ap.add_argument("--out", default=None, help="output directory (default results/)")
    args = ap.parse_args()

    outdir = pathlib.Path(args.out) if args.out else RESULTS
    outdir.mkdir(parents=True, exist_ok=True)
    stamp = dt.datetime.now().strftime("%Y-%m-%d_%H%M")

    if args.port:
        lines = capture(args.port, args.sweeps)
        raw = outdir / f"channel_survey_{stamp}.raw"
        raw.write_text("\n".join(lines) + "\n")
        print("saved", raw)
        base = raw.with_suffix("")
    else:
        raw = pathlib.Path(args.infile)
        lines = raw.read_text().splitlines()
        base = outdir / raw.stem

    rpd_occ, wifi, sweeps = parse(lines)
    score, wp = score_channels(rpd_occ, wifi)
    best10, best, plan_chs, guarded_found, half = recommend(score, args.guard, args.plan,
args.allow_above_ism)

    with open(base.with_suffix(".csv"), "w") as fh:
        fh.write("ch,freq_mhz,rpd_occupancy,wifi_penalty,score,in_ism\n")
```

```
        for c in range(NUM_CH):

fh.write(f"{c},{nrf_freq_mhz(c):.0f},{rpd_occ[c]:.4f},{wp[c]:.1f},{score[c]:.4f},{int(c <=
ISM_TOP_CH)}\n")
    label = args.label or raw.stem
    plot(rpd_occ, wifi, score, best, plan_chs, base.with_suffix(".png"), label)
    print("wrote", base.with_suffix(".csv"), "and", base.with_suffix(".png"))

    print(f"\nRPD sweeps: {sweeps}   WiFi APs seen: {len(wifi)}")
    by_wch: dict[int, list[int]] = {}
    for _s, _b, wch, rssi in wifi:
        by_wch.setdefault(wch, []).append(rssi)
    for wch in sorted(by_wch):
        r = by_wch[wch]
        print(f"  WiFi ch {wch:2d} ({wifi_center_mhz(wch):.0f} MHz): {len(r)} AP(s), strongest
{max(r)} dBm"
              f"  -> nRF ch {int(wifi_center_mhz(wch) - 2400 -
WIFI_HALF_BW_MHZ)}..{int(wifi_center_mhz(wch) - 2400 + WIFI_HALF_BW_MHZ)}")

    print("\nQuietest channels (score = 0.5*WiFi overlap + 0.5*RPD occupancy, lower is
better):")
    for c in best10:
        print(f"  ch {c:3d}  {nrf_freq_mhz(c):.0f} MHz  score={score[c]:.3f}
rpd={rpd_occ[c]*100:5.1f}%  wifi={wp[c]:8.1f}")
    if guarded_found:
        print(f"\nRECOMMENDED: ch {best} ({nrf_freq_mhz(best):.0f} MHz) — quietest channel
whose ±{half} neighbours are also quiet")
    else:
        print(f"\nRECOMMENDED: ch {best} ({nrf_freq_mhz(best):.0f} MHz) — quietest single
channel; "
              f"WARNING no channel had ±{half} quiet neighbours, expect adjacent-channel
interference")
    print(f"GROUP PLAN ({args.plan} links, ≥{args.guard} ch apart): {plan_chs}")
    if not args.allow_above_ism:
        above = [c for c in range(ISM_TOP_CH + 1, NUM_CH) if score[c] <= score[best]]
        if above:
            print(f"(channels above the ISM band that look quieter, NOT recommended for the
product: {above[:8]}...)")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```